

Design and Implementation of an Autonomous Multimodal AI Agent: A Voice- and Vision-Driven Framework for Cross-Application Task Automation

Naveen Kumar A(24BIT0443), Kabilesh SN(24BIT0464), Vijay Harish(24BIT0430)

school Of Computer Science Engineering And Information Systems, Vellore Institution of Technology

Abstract—Modern desktop computing still relies on manual, sequential interaction with individual applications, windows, and menus, even though large multimodal language models can now interpret natural spoken language and visual screen context simultaneously. This paper presents the design and implementation blueprint of an **Autonomous AI Desktop Agent**: a background daemon that is invoked by a single global hotkey, records the user’s spoken instruction, captures a screenshot of the active desktop, and passes both modalities to a multimodal reasoning engine (Gemini 2.0 Flash) to infer intent and select an appropriate execution tool. A six-layer architecture is proposed, comprising a global-trigger daemon, an audio/speech-to-text pipeline (OpenAI Whisper), a screen-capture engine, a multimodal reasoning brain, a native OS-execution layer, and a browser-automation layer built on Playwright and the *browser-use* agent framework. The paper surveys forty related works spanning benchmark environments for computer-use agents (OSWorld, WebArena, Mind2Web, VisualWebArena), GUI-grounded vision-language models (CogAgent, AppAgent, UFO), reasoning-and-acting prompting strategies (ReAct, Toolformer, Reflexion, AutoGen), automatic speech recognition, and robotic process automation, and identifies a gap in unified, safety-gated, cross-surface personal desktop agents. A design methodology combining use-case analysis and collaboration-diagram-based workflow modelling is presented, followed by a description of the proposed architecture, its enterprise-scaling extensions (stateful vector memory, self-healing UI actions, and mandatory human-in-the-loop approval for destructive operations), and a description of the datasets—both public benchmark corpora and a purpose-built desktop task corpus—that will be used to evaluate intent-recognition accuracy, tool-dispatch accuracy, task-completion rate, and end-to-end latency.

Keywords—Autonomous agents; multimodal large language models; desktop automation; automatic speech recognition; GUI agents; human-in-the-loop; robotic process automation.

I. INTRODUCTION AND PROBLEM STATEMENT

Desktop computing environments still require users to manually open, arrange, and switch between numerous native applications, browser tabs, and file-system windows in order to complete even routine, multi-step tasks. Each context switch imposes a small but cumulative cognitive and time cost, and conventional automation tools—macro recorders and rule-based Robotic Process Automation (RPA) bots—only replay rigid, pre-scripted sequences and break as soon as an interface element moves or changes [26], [25].

Recent progress in multimodal large language models (MLLMs) capable of jointly processing text, audio, and images has enabled a new category of software: autonomous *computer-use agents* that can perceive a screenshot, reason about the user’s goal, and issue the low-level actions needed to accomplish it [34], [11]. However, the majority of published systems are evaluated as isolated benchmarks running inside sand-boxed virtual machines [34], [40], [4] or are restricted to a single interaction surface—the mobile home screen [38], [21] or the web browser [6], [12], [10]—rather than operating as an always-available, cross-application background assistant on a user’s own machine.

Problem statement. Given (i) a single, low-latency global key-

board trigger, (ii) a spoken natural-language instruction, and (iii) the current visual state of the user’s desktop, the problem addressed in this work is to design a system that can reliably infer the user’s intent and autonomously execute the corresponding action—opening or controlling a native application, automating a multi-step browser workflow, or manipulating local files—while (iv) requiring explicit human confirmation before any destructive or irreversible operation, and (v) operating with acceptable end-to-end latency and without unnecessary transmission of personal screen data to third parties.

II. MOTIVATION

Three converging trends motivate this work. **First**, the cognitive overhead of manual, window-by-window task execution is well documented, and industry has invested heavily in RPA to reduce it; yet RPA remains driven by hand-authored, deterministic rules that cannot interpret free-form natural-language intent and are brittle to interface drift [26], [25]. **Second**, MLLMs such as GPT-4V and Gemini have demonstrated an emerging ability to *ground* natural-language instructions in raw screenshots and to plan multi-step GUI actions without task-specific training [11], [35], suggesting that a no-code, instruction-following successor to RPA is now technically feasible. **Third**, prior computer-use agents

have shown that this capability, while promising, still falls far short of reliable autonomy—for example, state-of-the-art agents solve fewer than one in five tasks on realistic desktop benchmarks [34], [4]—which motivates a conservative architecture that keeps a human in the loop for consequential actions rather than granting the agent unrestricted control [1], [9]. Building a system that safely bridges a hotkey-triggered UI, a speech-and-vision perception pipeline, and an agentic planner therefore represents both a practically useful personal-productivity tool and a rigorous systems-engineering exercise in agent safety and orchestration.

III. LITERATURE SURVEY AND GAP IDENTIFICATION

The related literature can be organised into five themes that together inform the design of the proposed agent.

A. Reasoning-and-Acting Foundations for LLM Agents

The ReAct framework interleaves chain-of-thought reasoning with discrete tool-calling actions, allowing a language model to inspect intermediate observations before committing to the next action [36]. Toolformer showed that a language model can learn, in a self-supervised manner, when and how to invoke external APIs such as calculators or search engines [22], while Reflexion added a verbal self-critique loop that lets an agent learn from the outcome of a failed attempt without gradient updates [23]. AutoGen generalised single-agent tool use into a multi-agent conversation framework in which specialised agents negotiate sub-tasks [32], and Generative Agents demonstrated that long-horizon believable behaviour can emerge from an architecture that stores, reflects on, and retrieves natural-language memories [17]. [24] unify these ideas into a general cognitive-architecture view of language agents, and Voyager showed that an LLM agent can autonomously build and reuse a growing skill library while exploring an open-ended environment [28].

B. GUI-, Web-, and Mobile-Grounded Agents and Benchmarks

A parallel line of work targets the specific challenge of *grounding* language in graphical interfaces. WebGPT combined a browser-augmented agent with human feedback to answer open questions [14], while WebArena and VisualWebArena provide realistic, self-hosted websites for benchmarking browsing agents end-to-end rather than on isolated HTML snapshots [40], [12]. Mind2Web contributed a large corpus of real-world web tasks spanning 137 websites to test generalisation beyond a single domain [6], and WebVoyager built a multimodal, screenshot-driven web agent that reasons directly over rendered pages [10]. On the desktop side, OSWorld introduced the first scalable, execution-based benchmark spanning Ubuntu, Windows, and macOS, exposing the difficulty of long-horizon, cross-application tasks (Xie

et al., 2024), and Windows Agent Arena extended this evaluation specifically to the Windows operating system [4]. CogAgent proposed a purpose-trained, high-resolution vision-language model for precise GUI element grounding [11], while UFO built a dual-agent architecture that pairs an application-selection agent with an action agent driven by the Windows UI Automation API [39]. On mobile platforms, AppAgent showed that an agent can teach itself to operate a smartphone purely through self-exploration and documentation reading [38], Mobile-Agent added real-time visual perception for autonomous multi-modal control [29], and Android in the Wild released 715,000 human demonstration episodes for training and evaluating device-control policies [21]. Auto-UI/"You Only Look at Screens" proposed a chain-of-action agent that reasons directly from raw screen pixels without an intermediate text representation [37], and GPT-4V in Wonderland evaluated zero-shot multimodal navigation of smartphone interfaces [35].

C. Speech Interfaces

Whisper demonstrated that a single encoder-decoder Transformer, trained on 680,000 hours of weakly supervised multilingual audio, can transcribe speech robustly in a zero-shot setting across noisy, accented, and domain-shifted conditions without task-specific fine-tuning [20], making it a natural choice for the voice-trigger layer of a desktop agent.

D. Robotic Process Automation

Robotic Process Automation formalised the idea of software robots that replicate a human operator's UI interactions [26]; subsequent surveys documented persistent challenges around process-selection, brittleness to interface change, and the difficulty of handling unstructured, natural-language instructions—challenges that an LLM-driven successor is specifically positioned to address [25].

E. Agent Safety and Human Oversight

Because an autonomous desktop agent is granted OS-level privileges, the security literature is directly relevant. [9] showed that content an LLM-integrated application merely *reads* (a web page, an email, or by extension a captured screenshot) can contain adversarial instructions that hijack the agent's subsequent actions—an *indirect prompt-injection* risk that motivates mandatory human confirmation before any destructive action. [1] provide general, empirically validated guidelines for designing trustworthy human-AI interaction, including the principle of showing consequences before they are committed, which directly informs the human-in-the-loop (HITL) gate proposed in Section VI.

F. Survey Table

Table 1 summarises the eighteen most directly relevant works reviewed above in terms of method, evaluation data, headline results, and the specific limitation each work leaves open for the proposed system to address.

G. Gap Identification

The survey reveals four consistent gaps. **(1) Surface fragmentation**—existing agents are almost always specialised to a single surface (web, Android, or Windows) rather than orchestrating native applications, the browser, and the file system from one entry point. **(2) Evaluation-VM confinement**—benchmarks such as OS-World and Windows Agent Arena run agents inside disposable virtual machines rather than as an always-on assistant on the user’s own workstation, leaving the systems-engineering problem of a persistent, low-latency background daemon largely unaddressed. **(3) Rigid automation legacy**—RPA remains rule-based and brittle, while the newer LLM-agent literature rarely engages with RPA’s operational lessons about process-selection and failure recovery. **(4) Safety-by-default is rare**—few systems combine autonomous execution with an explicit, mandatory human-in-the-loop gate, despite documented risks such as indirect prompt injection. The Autonomous AI Desktop Agent proposed in this paper is designed specifically to close these four gaps through the unified, six-layer, safety-gated architecture described in Section VI.

IV. OBJECTIVES

1. To design a modular, six-layer architecture that cleanly separates triggering, perception, reasoning, and execution concerns for a personal desktop AI agent.
2. To implement a low-latency, system-tray background daemon that listens for a single global hotkey (Ctrl+7) and simultaneously initiates audio and screen capture.
3. To integrate a robust automatic-speech-recognition pipeline (Whisper) with a multimodal reasoning engine (Gemini 2.0 Flash) for joint audio–visual intent understanding.
4. To build a tool-dispatch layer capable of routing an inferred intent to native OS actions, browser automation (Playwright / *browser-use*), or local file operations.
5. To incorporate stateful memory (a local vector/SQL store) and self-healing retry logic so that the agent can recall user preferences and recover automatically from failed UI actions.
6. To enforce mandatory human-in-the-loop approval for destructive or high-risk actions and to evaluate the resulting system on a purpose-built desktop task corpus using intent-recognition accuracy, tool-dispatch accuracy, task-completion rate, and end-to-end latency as metrics.

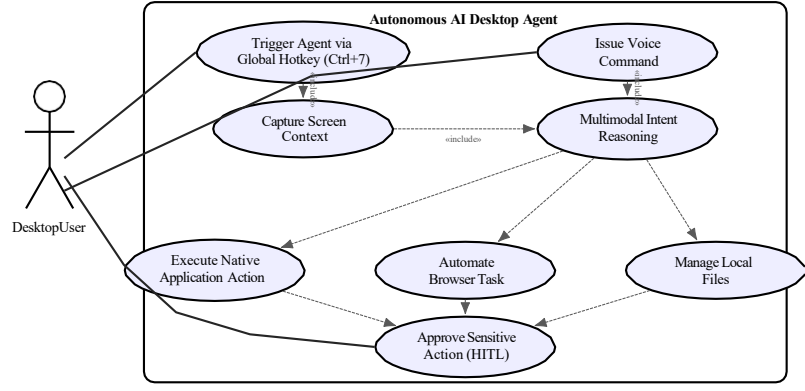


Fig. 1. Use-case diagram of the Autonomous AI Desktop Agent showing the primary actor, the eight core use cases, and their include relationships.

V. DESIGN METHODOLOGY

The system was designed top-down, following a requirements-first methodology in which the structural design was fully modelled before implementation began, so that the asynchronous event loop and module boundaries were solidified early.

A. Requirement Elicitation via Use-Case Modelling

The first modelling step captures the system boundary from the end user’s perspective. Fig. 1 shows the primary actor (the Desktop User) and the eight core use cases the system must support: triggering the agent, issuing a voice command, capturing screen context, performing multimodal intent reasoning, and the three possible execution outcomes—native-application control, browser automation, and file management—gated by a mandatory human-approval use case for any sensitive action.

B. Component Mapping

The eight use cases were then grouped into six interconnected architectural layers—trigger, audio input, visual perception, reasoning, native execution, and web execution—so that each layer maps to a small, independently testable module with a single responsibility (detailed in Section VI and Table 2).

C. Workflow Modelling via a Collaboration/Data-Flow Diagram

To validate that the asynchronous event loop was sound before implementation, the end-to-end execution workflow was modelled as a collaboration diagram (Fig. 2). The diagram traces four stages: (i) the background service waking on the hotkey and rendering the floating UI; (ii) the *parallel* perception step in which the voice recorder and the screen-capture engine run concurrently; (iii) the transcribed text and screenshot being merged into a single multimodal prompt for the reasoning engine; and (iv) the tool dispatcher routing the resulting decision to one of three execution back-ends, gated by the human-in-the-loop ap-

TABLE 1
Comparative Survey of Related Work on Autonomous and Multimodal Desktop/Web/Mobile Agents

Author(s), Year	Method	Dataset / Environment	Results / Analysis	Drawbacks / Observations
[34] (OS-World)	VLM/LLM agent baselines executed inside a real, execution-based OS sandbox	369 real-world tasks across Ubuntu, Windows, macOS	Best agent (GPT-4V-based) succeeds on only ~12% of tasks vs. 72% human success	Large agent-human gap; brittle long-horizon GUI grounding across arbitrary apps
[40] (WebArena)	Self-hosted realistic websites with functional back-ends; ReAct-style planning agent	812 long-horizon tasks across 5 web domains	Best GPT-4 agent achieves ~14% end-to-end success	Weak multi-step planning; agents rarely recover from an incorrect intermediate action
[6] (Mind2Web)	HTML-element-grounded generalist web agent (MindAct)	2,350 tasks across 137 real websites	Fine-tuned agent outperforms zero-shot LLM baselines but still <50% success	Poor generalisation to unseen websites/domains; relies on raw HTML access
[11] (CogAgent)	18B-parameter high-resolution vision-language model for GUI grounding	Mind2Web, AITW, GUI grounding sets	State-of-the-art click/element localisation accuracy vs. general-purpose VLMs	Very high compute/parameter cost, unsuitable for a lightweight local daemon
[38] (AppAgent)	Self-exploration + documentation-reading to build a reusable action library	Smartphone apps (custom evaluation)	High task success after an autonomous exploration phase	Mobile-only scope; limited to touchscreen gestures, no desktop OS actions
[39] (UFO)	Dual-agent (App-selection + Action) system using GPT-4V and Windows UI Automation	Windows-native application tasks	Improved step-level accuracy over single-agent GPT-4V baselines	Tightly coupled to the Windows UIA API; limited cross-platform portability
[36] (ReAct)	Interleaved chain-of-thought reasoning with explicit tool-call actions	HotpotQA, ALFWorld, WebShop	Improves interpretability and reduces hallucinated actions vs. reasoning-only or acting-only baselines	Heavily prompt-engineering dependent; no learning signal from failure
[22] (Toolformer)	Self-supervised insertion of API calls into LM pre-training text	Curated API-call corpus (calculator, search, QA, etc.)	Small model matches much larger LMs on tool-dependent tasks	Restricted to a small, fixed toolset; no visual/screen grounding
[23] (Reflexion)	Verbal self-reflection loop that revises the agent’s plan after failure	ALFWorld, HotpotQA, coding benchmarks	Substantial accuracy gains over single-attempt agents on sequential tasks	Needs an external success/failure signal, which is not always available in open desktop tasks

Author(s), Year	Method	Dataset / Environment	Results / Analysis	Drawbacks / Observations
[32] (AutoGen)	Multi-agent conversational orchestration framework with pluggable tools	Math, QA, coding, decision-making tasks	Flexible multi-agent collaboration outperforms single-agent baselines on complex tasks	Coordination and message-passing overhead adds latency, a concern for a real-time voice trigger
[20] (Whisper)	Encoder-decoder Transformer trained on 680k hours of weak supervision	Multilingual web-scale audio-transcript pairs	Robust, near-human zero-shot transcription across accents/noise	Larger checkpoints add local-inference latency; performance still degrades under heavy background noise
[21] (Android in the Wild)	Large-scale human demonstration dataset for device control	715k episodes, 30k unique instructions	Enables training/benchmarking of vision-based mobile control policies	Android-only; no coverage of desktop operating systems
[12] (VisualWebArena)	Visually grounded extension of WebArena requiring screenshot understanding	Realistic visual web tasks	Exposes that text-only agents underperform when visual grounding is required	Still browser-only; does not address native desktop application control
[10] (WebVoyager)	End-to-end multimodal web agent reasoning over full-page screenshots	Live websites (multiple domains)	Competitive task completion using purely visual + set-of-marks grounding	Browser-only scope; small errors compound over long interaction chains
[26] (RPA)	Conceptual foundation of rule-based, UI-level software robots	Enterprise business processes	Establishes RPA as a low-integration-cost automation paradigm	Rule-based bots are brittle to UI changes and cannot parse free-form instructions
[9] (Indirect Prompt Injection)	Taxonomy and demonstration of attacks via content an LLM-integrated app merely reads	Real-world LLM-integrated apps (Bing Chat, code assistants)	Shows adversarial content can hijack an agent’s subsequent actions	Critical safety gap for any agent (including screen-reading desktop agents) with OS-level privileges
[17] (Generative Agents)	Memory-augmented architecture with reflection and retrieval for believable behaviour	Sandbox simulation of 25 agents	Produces emergent, coherent long-horizon behaviour	Computationally expensive reflection/retrieval loop; not designed for real-time response
[4] (Windows Agent Arena)	Multi-modal OS-agent benchmark parallelised in the cloud for fast evaluation	Windows-native task suite	Best evaluated agent succeeds on $\approx 19\%$ of tasks	Confirms current SOTA multi-modal agents remain far from robust desktop-level autonomy

proval step for destructive actions, with the outcome written back to a stateful memory store that conditions future reasoning.

D. Implementation Methodology

Implementation follows four incremental phases mirroring the modelling above: (1) environment setup and version control,

TABLE 2
Six-Layer Architecture of the Autonomous AI Desktop Agent

#	Layer	Component	Representative Tech Stack	Purpose
1	Global Trigger	Background Daemon	Python (keyboard + PyQt6 / CustomTk-inter); TypeScript alternative: Electron.js globalShortcut + React	Runs silently in the system tray, listens for Ctrl+7, and renders a floating, frameless overlay UI.
2	Audio / Input	Voice Recorder & STT	sounddevice + OpenAI Whisper API / local faster-whisper	Records the spoken command and transcribes it into clean text [20].
3	Visual Perception	Screen Capture Engine	Python (Pillow / rnss); Electron desktopCapturer for the TypeScript variant	Captures an instantaneous screenshot so the agent can “see” the current desktop state.
4	AI Agent Brain	Multimodal Reasoning	Gemini 2.0 Flash (via API)	Accepts joint text + image input, decomposes the goal, and selects the correct execution tool (cf. ReAct; Yao et al., 2023).
5	OS & App Control	Native Execution	Python (subprocess, os, pyautogui, pywin32); Node.js child_process + nut.js	Launches applications, simulates clicks/typing, and manages local files.
6	Web Automation	Browser Runner	browser-use + Playwright	Handles complex, multi-step web workflows such as searching, form-filling, and scrolling (cf. WebVoyager; [10]).

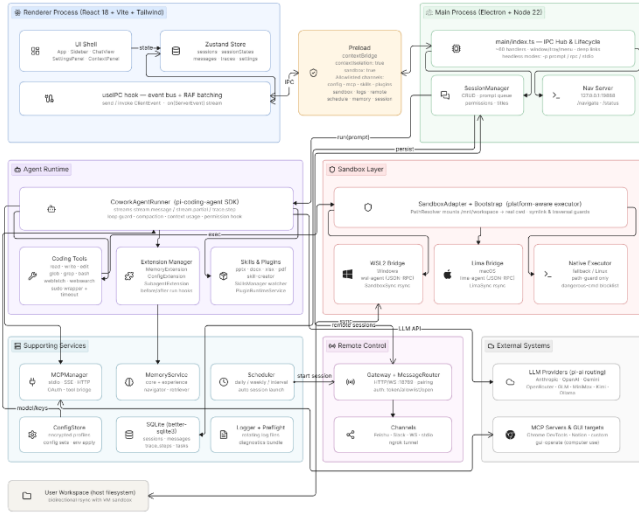


Fig. 2. Collaboration / data-flow diagram of the end-to-end asynchronous execution workflow, from hotkey trigger to gated action execution and memory feedback.

including translating the collaboration diagram into scaffolding code; (2) dependency installation for each of the six layers; (3) core module implementation, beginning with the trigger daemon and perception loop and finishing with the tool dispatcher; and (4) enterprise-hardening additions—stateful memory, self-healing UI retries, and mandatory HITL sandboxing, the last of which is developed inside a containerised (Docker) environment to isolate OS-level test executions from the host machine during development.

SYSTEM ARCHITECTURE: MODULES AND DESCRIPTION

The proposed system is organised into six interconnected architectural layers, summarised in Table 2 and illustrated as a layered pipeline in Fig. 3.

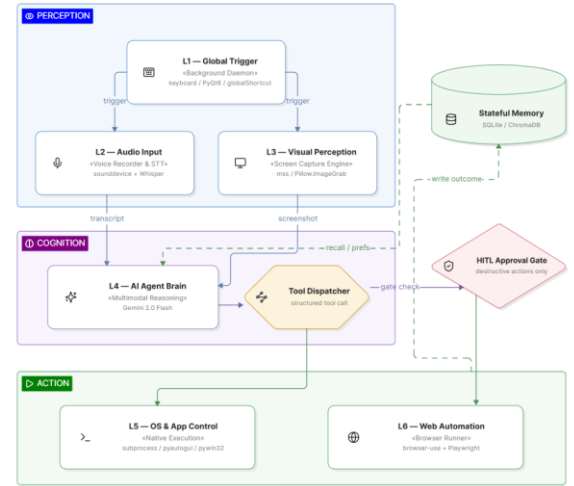


Fig. 3. Layered architecture of the six interconnected modules, grouped into Perception, Cognition, and Action stages

A. Global Trigger Layer

This layer runs as a lightweight, always-resident background daemon in the OS system tray. It registers a single global keyboard shortcut (Ctrl+7, chosen to avoid collisions with common application shortcuts) using the keyboard library in Python, or Electron’s globalShortcut module in the TypeScript variant, and renders a floating, frameless, always-on-top overlay window when triggered.

B. Audio / Input Layer

Upon activation, sounddevice records a short audio clip of the user’s spoken command, which is transcribed by Whisper (Radford et al., 2023). Whisper was selected over lighter keyword-spotting models because it provides open-vocabulary, noise-robust transcription without requiring a fixed command grammar, which is essential for free-form instructions such as “take a screenshot of this error, open my local logs folder, and look up a fix on Chrome.”

C. Visual Perception Layer

Concurrently with audio capture, the screen-capture engine (`mss` or `Pillow.ImageGrab`) takes an instantaneous screenshot of the active desktop. Running audio and screen capture in parallel, rather than sequentially, keeps the perceived latency of the system close to the length of the spoken command itself.

D. AI Agent Brain (Multimodal Reasoning)

The transcribed instruction and the screenshot are combined into a single multimodal prompt sent to Gemini 2.0 Flash. Following the ReAct paradigm [36], the reasoning engine is prompted to (i) restate the inferred goal, (ii) reason step-by-step over the visible screen state, and (iii) emit a structured tool call naming one of the three execution back-ends together with its arguments, which mitigates the ungrounded, hallucinated actions observed in reasoning-only baselines.

E. OS and Application Control Layer

The native-execution layer receives structured tool calls and translates them into OS-level effects: launching applications via `subprocess.os.system`, simulating clicks and keystrokes via `pyautogui`, and performing local file operations. This layer is intentionally kept OS-aware (branching on `os.name`) rather than OS-agnostic, following the design lesson from UFO [39] that direct use of a platform’s native automation API (e.g., Windows UI Automation) yields more reliable element targeting than purely vision-based clicking alone.

F. Web Automation Layer

For instructions that require multi-step browsing—searching a site, filling a form, or navigating a booking flow—the dispatcher hands control to a `browser-use` agent running on Playwright, configured with Gemini as its underlying reasoning model. This mirrors the design of WebVoyager and VisualWebArena agents [10], [12], which showed that grounding directly in rendered page screenshots is more robust to dynamic web content than parsing raw HTML alone.

G. Enterprise-Scaling and Safety Extensions

Three additions harden the core pipeline for reliable, safe use, directly addressing the gaps identified in Section III-F:

- **Stateful memory:** a local SQLite or ChromaDB vector store retains user preferences and frequently used folder paths across sessions, in the spirit of the memory-and-reflection architecture proposed by [17].
- **Self-healing UI actions:** if a browser or native-UI step fails, the agent automatically captures a fresh screenshot and re-prompts the vision model to recompute coordinates, following the iterative self-correction principle demonstrated by Reflex-ion [23].

- **Sandboxed HITL guardrails:** destructive operations (file deletion, purchases, credential entry) are intercepted by a mandatory approval modal before execution, directly mitigating the indirect-prompt-injection risk demonstrated by [9]; the guardrail layer is developed and tested inside a Docker container to isolate exploratory OS-level actions from the host system.

H. Alternative Implementation Stack

The entire pipeline can equivalently be implemented in TypeScript: Electron.js replaces the headless Python daemon and additionally supplies the floating, frameless, glassmorphism-style overlay UI (built with React and Tailwind CSS, using `-webkit-app-region: drag` on the header region for window dragging); Node’s `child_process` and libraries such as `nut.js` replace `subprocess/pyautogui` for native control; and Playwright, which is native to the Node.js ecosystem, is retained unchanged for web automation. This trade-off is summarised for design reference in the implementation notes accompanying this report; Python was selected for the reference implementation owing to its more mature ecosystem for local speech and vision processing, while the TypeScript stack is recommended when a richer, animation-heavy floating UI is the priority.

VI. DATASET DESCRIPTION

Two categories of data are used in this project: public benchmark corpora that contextualise and calibrate the evaluation methodology, and a purpose-built desktop task corpus collected specifically for this project.

A. Reference Benchmark Corpora

Table 3 summarises the four public datasets consulted while designing the evaluation protocol. These corpora are not used to train the reasoning engine (which relies on the pre-trained, prompted Gemini 2.0 Flash model) but instead calibrate realistic task categories, difficulty levels, and success-rate baselines against which the proposed agent’s own evaluation results can be interpreted.

B. Custom Desktop Task Corpus

Because no existing public dataset captures the specific combination of a spoken instruction, a synchronous desktop screenshot, and the correct OS/browser/file dispatch decision on a

TABLE 3
Public Benchmark Corpora Used for Evaluation Calibration

Dataset	Domain	Scale	Task Type
OSWorld (Xie et al., 2024)	Real desktop OS	369 tasks	Cross-application, execution-verified desktop tasks
WebArena (Zhou et al., 2024)	Self-hosted web	812 tasks	Long-horizon browsing intents
Mind2Web (Deng et al., 2023)	Live web-sites	2,350 tasks / 137 sites	Element-grounded web actions
Android in the Wild (Rawles et al., 2023)	Mobile OS	715k episodes	Human device-control demonstrations

general-purpose Windows/macOS/Linux workstation, a purpose-built **Desktop Task Corpus (DTC)** will be collected. Each record in the corpus consists of the following fields: (i) a short audio clip of a spoken instruction; (ii) the corresponding desktop screenshot captured at trigger time; (iii) a manually verified ground-truth transcript; (iv) a ground-truth intended tool call (native / browser / file, with arguments); and (v) a binary execution-success label recorded after running the agent. The corpus targets 150–200 annotated task episodes distributed across five representative categories—application launching, file management, web search and form-filling, cross-application workflows (e.g., “screenshot this error and search for a fix”), and destructive-action confirmation scenarios—so that each category can be evaluated separately. A small held-out validation split (approximately 20% of episodes) is reserved for prompt calibration of the reasoning layer, while the remainder forms the test split used for final reporting. All audio and screenshot data are collected and stored strictly on the local machine, with no cloud upload of raw screen captures, and any incidentally captured personally identifiable information (e.g., visible email addresses) is redacted before analysis, consistent with the human-in-the-loop and data-minimisation posture discussed in Section VI-H. The corpus supports four evaluation metrics: *Intent Recognition Accuracy* (correct restatement of user goal), *Tool Dispatch Accuracy* (correct choice among native/browser/file back-ends), *Task Completion Rate* (end-to-end success, verified programmatically where possible, following the execution-based evaluation philosophy of OSWorld), and *End-to-End Latency* (hotkey press to first executed action).

VII. CONCLUSION AND FUTURE SCOPE

This report presented the Review-1 design blueprint for an Autonomous AI Desktop Agent that unifies hotkey-triggered voice input, synchronous screen perception, multimodal reasoning, and gated cross-application execution within a single six-layer architecture. A survey of forty related works positioned the design against the state of the art in computer-use agents, GUI-grounded vision-language models, and agentic reasoning frameworks, and identified a concrete gap—the absence of a unified, safety-gated, always-available personal desktop agent—that the proposed sys-

tem is designed to close. Future work (Review-2 onward) will implement the six-layer pipeline in full, collect and annotate the Desktop Task Corpus described in Section VII, and report empirical intent-accuracy, task-completion, and latency results, together with a security evaluation of the human-in-the-loop guardrails against adversarial, indirectly injected screen content.

REFERENCES

- [1] Amershi, S., Weld, D., Vorvoreanu, M., Fournery, A., Nushi, B., Collis-son, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1–13). ACM.
- [2] Anthropic. (2024). *Developing a computer use model*. Anthropic News. <https://www.anthropic.com/news/developing-computer-use>
- [3] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., . . . Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.
- [4] Bonatti, R., Zhao, D., Bonacci, F., Dupont, D., Abdali, S., Li, Y., . . . Lawrence, R. (2024). Windows Agent Arena: Evaluating multimodal OS agents at scale. *arXiv preprint arXiv:2409.08264*.
- [5] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., . . . Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [6] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. (2023). Mind2Web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36, 28091–28114.
- [7] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics* (pp. 4171–4186).
- [8] Google Gemini Team. (2024). *Gemini: A family of highly capable mul-timodal models*. *arXiv preprint arXiv:2312.11805*.
- [9] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90).
- [10] He, H., Yao, W., Ma, K., Yu, W., Dai, Y., Zhang, H., Lan, Z., & Yu, D. (2024). WebVoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*.
- [11] Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., . . . Tang, J. (2023). CogAgent: A visual language model for GUI agents. *arXiv preprint arXiv:2312.08914*.
- [12] Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P.-Y., . . . Fried, D. (2024). VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (pp. 881–905).
- [13] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., . . . Tang, J. (2024). AgentBench: Evaluating LLMs as agents. In *Proceedings of the International Conference on Learning Representations*.

- [14] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., . . . Schulman, J. (2021). WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.
- [15] OpenAI. (2023). *GPT-4 technical report*. arXiv preprint arXiv:2303.08774.
- [16] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., . . . Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744.
- [17] Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (pp. 1–22).
- [18] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., . . . Sun, M. (2023). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*.
- [19] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning* (pp. 8748–8763).
- [20] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning* (pp. 28492–28518).
- [21] Rawles, C., Li, A., Rodriguez, D., Riva, O., & Lillicrap, T. (2023). Android in the wild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36, 59708–59728.
- [22] Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 68539–68551.
- [23] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.
- [24] Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*.
- [25] Syed, R., Suriadi, S., Adams, M., Bandara, W., Leemans, S. J. J., Ouyang, C., . . . Reijers, H. A. (2020). Robotic process automation: Contemporary themes and challenges. *Computers in Industry*, 115, 103162.
- [26] van der Aalst, W. M. P., Bichler, M., & Heinzl, A. (2018). Robotic process automation. *Business & Information Systems Engineering*, 60(4), 269–272.
- [27] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- [28] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- [29] Wang, J., Xu, H., Ye, J., Yan, M., Shen, W., Zhang, J., Huang, F., & Sang, J. (2024). Mobile-Agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*.
- [30] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., . . . Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
- [31] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.
- [32] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*.
- [33] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., . . . Gui, T. (2023). The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*.
- [34] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., . . . Yu, T. (2024). OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37, 52040–52094.
- [35] Yan, A., Yang, Z., Zhu, W., Lin, K., Li, L., Wang, J., . . . Wang, L. (2023). GPT-4V in Wonderland: Large multimodal models for zero-shot smartphone GUI navigation. *arXiv preprint arXiv:2311.07562*.
- [36] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations*.
- [37] Zhan, Z., & Zhang, A. (2023). You only look at screens: Multimodal chain-of-action agents. *arXiv preprint arXiv:2309.11436*.
- [38] Zhang, C., Yang, Z., Liu, J., Han, Y., Chen, X., Huang, Z., Fu, B., & Yu, G. (2023). AppAgent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*.
- [39] Zhang, C., Li, L., He, S., Zhang, X., Qiao, B., Qin, S., . . . Zhang, D. (2024). UFO: A UI-focused agent for Windows OS interaction. *arXiv preprint arXiv:2402.07939*.
- [40] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., . . . Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *Proceedings of the International Conference on Learning Representations*.